

RAPPORT DE PROJET TRAITEMENT DE DONNÉES DE PREMIÈRE ANNÉE

Élèves 1A

A REMPLIR

Étudiants :

Groupe à préciser

Ilan GUEDJ

Jules FRULLI

Thomas ETIENNE

Alexandre FRATTER-BARDY

Responsable :

Johann FAOUZI

Encadrant :

Clément VALOT

Table des matières

Introduction	1
1 Modélisation et Diagrammes UML	2
2 Architecture et choix techniques	3
2.1 Les outils utilisés	3
2.2 La programmation orientée objet	3
2.3 Les choix d'utilisations	3
3 Implémentation de la programmation orientée objet (POO)	5
3.1 Les classes modèles	5
3.2 Les classes Loaders	5
3.3 Les classes services	6
3.4 Les avantages de cette architecture logicielle	6
4 Analyse des bases de données et traitements	7
4.1 Présentation globale des données	7
4.2 Analyse détaillée d'un jeu de données : le cas du Basketball	7
4.2.1 Structure et types de variables	7
4.2.2 Nettoyage et gestion des valeurs manquantes	7
4.2.3 Transformation spécifique : conversion des mesures impériales	7
4.3 L'harmonisation	8

Introduction

Suite au succès des Jeux Olympiques et Paralympiques de Paris 2024, la gestion et l'analyse des données sportives sont devenues des enjeux stratégiques majeurs. C'est dans ce contexte stimulant que le Ministère des Sports, de la Jeunesse et de la Vie Associative souhaite développer une nouvelle application dédiée à la gestion centralisée des résultats de compétitions sportives. Cependant, l'histoire récente des grands projets informatiques de l'État est marquée par des échecs très coûteux (tels que les logiciels Scribe ou les systèmes de paie de l'Éducation nationale). Par conséquent, les attentes en matière de fiabilité, de conception et de résultats pour cette nouvelle application sont exceptionnellement élevées.

L'objectif principal est de concevoir et de développer une application robuste capable de modéliser l'écosystème sportif à travers la création d'entités telles que des matchs, des compétitions, des équipes ou encore des joueurs. L'application doit non seulement sauvegarder les caractéristiques pertinentes de ces acteurs, mais aussi offrir la possibilité d'intégrer dynamiquement les résultats de nouvelles compétitions afin de mettre à jour leurs statistiques de manière automatisée. Une des complexités majeures de ce cahier des charges réside dans l'exigence de modularité : l'application doit pouvoir fonctionner sur plusieurs jeux de données différents. Si certaines caractéristiques demeurent spécifiques à une discipline, le socle central des fonctionnalités se doit d'être universel et opérationnel indépendamment du sport traité.

Pour ce faire, nous devons manipuler divers jeux de données brutes. Tout l'enjeu a été de concevoir un système capable d'ingérer, de nettoyer et de structurer ces informations afin de les rendre exploitables par notre application, quel que soit le contexte sportif.

Afin de répondre efficacement à ces enjeux de modélisation et de flexibilité, nous avons raisonné en programmation orientée objet. Cette approche s'est révélée particulièrement pertinente, car elle permet de traduire fidèlement les concepts du monde réel en objets informatiques interagissant entre eux. Enfin, pour répondre à la contrainte stricte de maintenabilité imposée par le Ministère, une attention toute particulière a été accordée à l'architecture logicielle. L'application a été codée dans le strict respect des bonnes pratiques de programmation, garantissant ainsi un code propre, documenté et évolutif, prêt à être repris et enrichi par de futures équipes de développeurs sans risquer l'impasse technique.

1 Modélisation et Diagrammes UML

2 Architecture et choix techniques

2.1 Les outils utilisés

Le langage de programmation utilisé lors de ce projet est *Python 3*. La bibliothèque *Pandas* est au cœur du traitement des fichiers CSV. Elle nous permet de réaliser toutes les tâches dont nous avons besoin pour répondre au cahier des charges : lecture, filtrage et sauvegarde des données. De plus, nous avons utilisé la bibliothèque *NumPy* pour optimiser les calculs des statistiques sur de grandes séries de données, par exemple pour le calcul de moyennes ou d'indicateurs clés sur les joueurs. Concernant la visualisation des données, la bibliothèque *Matplotlib* a été utilisée pour sa capacité à générer des graphiques, qu'ils soient complexes ou non, comme des histogrammes ou des heatmaps. Enfin, l'utilisation des modules natifs *os* et *sys* permettent de garantir la portabilité du code : gestion des chemins relatifs des fichiers de données et ce, peu importe l'ordinateur sur lequel le script est exécuté.

2.2 La programmation orientée objet

Lors de la réalisation de ce projet, nous avons fait face à un problème de données. En effet, ces données font référence à des sports différents. Ces données sont donc, par nature, hétérogènes. Nous avons par exemple des données sur le football, d'autres sur le tennis. Mais ces dernières sont séparées selon le genre de l'athlète. Par ailleurs, les fichiers CSV n'ont ni les mêmes colonnes, ni les mêmes unités de mesure d'un sport à l'autre. Ainsi, une approche avec de simples listes ou dictionnaires aurait créé un code peu lisible, rempli de conditions sur le sport choisi. Grâce à la programmation orientée objet (POO), nous avons créé une manière universelle de représenter nos sports, matchs et athlètes. Peu importe comment le joueur est décrit dans le fichier CSV, les loaders extraient la donnée et fabriquent un objet *Player* avec des attributs universels comme par exemple la taille ou le nom. Aussi, les responsabilités sont séparées, le fichier *main.py* ne fait que gérer le menu, les loaders ne font que traduire les CSV en objets et les différents services proposés ne font que générer des graphiques ou des statistiques à partir de ces objets.

```
try:
    if "-" in h_str or "'" in h_str:
        parts = h_str.replace("'", "-").split("-")
        if len(parts) == 2:
            pieds = float(parts[0])
            pouces = float(parts[1])
            h_cm = (pieds * 30.48) + (pouces * 2.54)
            if h_cm > 140:
                tailles.append(h_cm)
```

Cela représente un extrait de notre code portant sur le nettoyage des données. Nous suivons une logique de standardisation de la taille de l'objet *Player*, nous convertissons ainsi les données impériales (pieds-pouces) en système métrique (cm).

2.3 Les choix d'utilisations

Nous avons fait le choix d'utiliser *Pandas* et non le module CSV natif. En effet, *Pandas* permet de gérer facilement les données textuelles incomplètes. *Pandas* permet également de pouvoir filtrer les données de manière plus efficace que si nous n'avions pas utilisé cette bibliothèque.

Par exemple, dans notre classe *DataUpdater*, pour trouver un joueur quand on ne sait pas si la colonne s'appelle *fistr_name* ou *player_name*, Pandas fusionne virtuellement les colonnes des noms et applique un filtre de recherche.

```
# On fusionne toutes les colonnes de noms trouvées
noms_complets_virtuels = df[colonnes_noms].astype(str).agg(' '.join, axis=1)

masque = noms_complets_virtuels.str.contains(nom_recherche, case=False, na=False)
resultats = df[masque]
```

Pour répondre au cahier des charges, nous avons également utilisé la bibliothèque Matplotlib qui nous permet de réaliser des graphiques complets comme par exemple l'analyse de la taille des joueurs avec le pourcentage de victoires. Pour représenter la corrélation entre ces deux variables, nous avons utilisé la fonction *plt.hexbin()*. En effet, quand on croise deux variables comme la taille et le pourcentage de victoires pour des milliers d'individus, un graphique classique crée un nuage de points illisible. Donc cette fonction regroupe les joueurs par alvéoles et utilise la couleur pour montrer la concentration des individus dans une certaine zone. Cela offre une lecture plus fluide.

```
# Carte de chaleur avec un dégradé de Jaune (peu de joueurs) à Rouge (beaucoup de joueurs)
heatmap = plt.hexbin(tailles_valides, win_rates_valides, gridsize=15, cmap='YlOrRd', mincnt=1)
```

La mise à jour des données est isolée dans la classe *DataUpdater* car dans le cadre d'un outil utilisé par plusieurs acteurs, modifier les données en mémoire vive ne sert à rien si ce n'est pas sauvegardé. Ainsi, cette classe garantit que toute modification des données ou tout ajout de nouvelles données est réécrite de manière sécurisée et que les Loaders sont rechargés par la suite pour que l'application soit toujours à jour.

3 Implémentation de la programmation orientée objet (POO)

Afin de gérer la diversité de nos données sportives et d'éviter un code difficile à maintenir, notre application repose sur une architecture logicielle modulaire. Nous avons divisé notre code en trois grandes familles de classes, respectant ainsi le principe de responsabilité unique : les modèles pour encapsuler la donnée, les loaders pour extraire et nettoyer, et les services pour traiter et analyser.

3.1 Les classes modèles

Nous avons créé des classes spécifiques comme *Player*, *Team* et *Sport* qui agissent comme des conteneurs de données. Au lieu de manipuler directement des lignes de DataFrames ou des dictionnaires tout au long du code, chaque élément pertinent de nos fichiers CSV est transformé en un objet. L'objectif principal de cette démarche est la standardisation et la sécurité des données. Par exemple, qu'un athlète provienne du fichier *atp_players_2024.csv* (tennis) ou *player.csv* (football), il devient un objet *Player* avec des attributs stricts et communs comme *prenom_nom*, *taille*, ou *genre*. Cette encapsulation garantit que le reste du programme n'a pas à se soucier des noms de colonnes originaux du CSV. Si une clé de dictionnaire vient à changer dans les données brutes, l'erreur ne se propagera pas dans toute l'application : la donnée est verrouillée et uniformisée au sein de l'objet. Cela permet à l'application de traiter n'importe quel sport de manière universelle.

```
for i in range(len(df_basketball)):
    prenom = df_basketball.loc[i, "first_name"]
    nom = df_basketball.loc[i, "last_name"]
    nom_complet = f"{prenom} {nom}"
    joueur = Player(None, None, None, None, None, None, None)

    joueur.prenom_nom = nom_complet
    joueur.nationalite = None
    joueur.date_naissance = df_basketball.loc[i, "birthdate"]
    joueur.genre = None
    joueur.height = df_basketball.loc[i, "height"]
    joueur.poids = df_basketball.loc[i, "weight"]
    joueur.team = df_basketball.loc[i, "team_id"]
```

FIGURE 3.1 : Légende optionnelle

3.2 Les classes Loaders

Pour protéger la logique de notre application des incohérences souvent présentes dans les jeux de données bruts, nous avons isolé la lecture des fichiers. La mission des classes *MatchLoader*, *PlayerLoader* et *TeamLoader* est d'ouvrir les fichiers CSV, de gérer les valeurs manquantes via des blocs *try/except*, de convertir les types de variables (par exemple, transformer une chaîne de caractères en entier) et de fabriquer les objets finaux. L'intérêt de cette approche est que chaque sport possède son propre comportement lors du chargement, sans impacter le programme principal. Par exemple, le loader du tennis gère la séparation des données en deux fichiers distincts (ATP pour les hommes et WTA pour les femmes). Le loader va lire ces deux fichiers séparément,

leur ajouter une colonne *genre* pour garder la trace de la catégorie, puis utiliser *Pandas* pour les concaténer. Il renvoie ainsi une liste unique et complète de joueurs de tennis prête à être analysée.

```
df_atp["genre"] = "Homme"
df_wta["genre"] = "Femme"

df_tennis = pd.concat([df_atp, df_wta], ignore_index=True)
```

FIGURE 3.2 : Légende optionnelle

3.3 Les classes services

Les services sont les classes qui effectuent les calculs complexes ou génèrent les affichages, sans stocker la donnée de manière permanente. Nous avons par exemple la classe *AnalysePhysiologique*, qui reçoit la liste des objets *Player* et s'occupe de toute la visualisation (génération des histogrammes et des heatmaps). Le fait de l'isoler dans une classe évite de surcharger le fichier principal **main.py** avec de multiples paramétrages Matplotlib et permet de réutiliser ces graphiques facilement. De la même manière, d'autres classes de services gèrent la logique métier de l'application. La classe *ChampionshipPointsCalculator* s'occupe du calcul des points de victoires ou de défaites. Puisque les règles de calcul diffèrent selon les sports, cette classe centralise cette complexité : le programme principal se contente d'appeler une méthode unique, et le service applique la bonne formule. Enfin, la classe *DataUpdater* gère l'aspect persistance des données. Elle isole les opérations d'écriture afin de s'assurer que toute modification manuelle ou importation de nouvelle compétition n'écrase pas accidentellement nos historiques.

```
class AnalysePhysiologique:
    """
    Classe responsable de l'analyse et de la visualisation
    des données morphologiques des joueurs.
    """
    def __init__(self, liste_joueurs, sport):
        self.liste_joueurs = liste_joueurs
        self.sport = sport
```

FIGURE 3.3 : Légende optionnelle

3.4 Les avantages de cette architecture logicielle

Cette architecture nous a demandé un effort de modélisation et de conception au départ, mais elle apporte des bénéfices majeurs pour le projet. Premièrement, la maintenabilité : le fichier principal *main.py* se contente d'afficher les menus et d'appeler des méthodes aux noms explicites, agissant uniquement comme un chef d'orchestre. En cas de bug lors d'un calcul, nous savons exactement dans quelle classe de service chercher, sans avoir à parcourir les lignes de code d'interface. Deuxièmement, l'application est évolutive. Si nous souhaitons ajouter un nouveau sport possédant ses propres règles d'importation, comme le rugby, il n'est pas nécessaire de modifier le système de menus, les algorithmes de calcul ou les générateurs de graphiques. Il suffira d'ajouter les fichiers CSV correspondants et de créer une simple classe *RugbyLoader*. Le reste de l'application s'adaptera automatiquement pour traiter ces nouvelles entités.

4 Analyse des bases de données et traitements

4.1 Présentation globale des données

Notre application traite des données issues de cinq sports différents : le football, le tennis, le basketball, le volleyball et League of Legends. D'autres bases de données concernant d'autres sports ont été données mais nous avons fait le choix de ne travailler qu'à partir de ces cinq premiers sports. Les données sont stockées au format CSV. Étant donné que ces jeux de données proviennent de sources distinctes, ils présentent une forte hétérogénéité. Les noms des colonnes diffèrent selon les sports, par exemple *first_name* et *last_name* pour le basket, contre *name_first* et *name_last* pour le tennis, tout comme les unités de mesure utilisées. Pour illustrer notre méthode de traitement, nous détaillerons ici l'analyse et le nettoyage du jeu de données du Basketball.

4.2 Analyse détaillée d'un jeu de données : le cas du Basketball

4.2.1 Structure et types de variables

Le fichier *player.csv* du Basketball recense les athlètes professionnels. Chaque ligne représente un joueur unique. Parmi les variables brutes à notre disposition, nous exploitons principalement : les variables textuelles telles que *first_name*, *last_name*, *team_name* et les variables numériques stockées sous format texte telles que *height* (taille) et *weight* (poids).

```
person_id,first_name,last_name,birthdate,height,weight,jersey,position,team_id
1630173,Precious,Achiuwa,1999-09-19,6-8,225,5,Forward,1610612761
203500,Steven,Adams,1993-07-2,6-11,265,4,Center,1610612763
1628389,Bam,Adebayo,1997-07-18,6-9,255,13,Center-Forward,1610612748
1630534,Ochai,Agbaji,2000-04-2,6-5,215,30,Guard,1610612762
```

FIGURE 4.1 : Légende optionnelle

4.2.2 Nettoyage et gestion des valeurs manquantes

Un jeu de données brut contient souvent des valeurs aberrantes ou manquantes. Lors de la phase de chargement effectuée par notre classe *BasketballPlayerLoader*, nous appliquons un nettoyage systématique. Nous utilisons les fonctionnalités de la bibliothèque Pandas pour filtrer ces données (par exemple via *.dropna()*) ou nous les gérons au niveau de l'extraction par des blocs *try/except*. Si un joueur ne possède ni taille ni poids valides dans le fichier source, une valeur par défaut de 0 ou None lui est attribuée temporairement, afin de ne pas faire planter les calculs statistiques globaux ou de l'exclure purement et simplement de l'affichage.

4.2.3 Transformation spécifique : conversion des mesures impériales

La particularité majeure de ce jeu de données est son système de mesure américain. La taille y est renseignée en pieds et pouces sous forme de chaîne de caractères (ex : "6-8" pour 6 pieds et 8 pouces). Pour que ces données puissent être comparées avec celles du football ou du volleyball, il fallait les convertir dans le système métrique. Nous avons donc implémenté un algorithme d'extraction et de conversion. Pour la taille, le code divise la chaîne de caractères grâce à la méthode *.split("-")*. Il multiplie ensuite la première valeur représentant les pieds par 30,48 et la seconde représentant les pouces par 2,54, avant d'additionner les deux pour obtenir la taille exacte en centimètres.

4.3 L'harmonisation

Ce travail de nettoyage et de transformation a été adapté à la spécificité de chaque jeu de données. À titre d'exemple, pour la base de données du Tennis, les données étaient scindées en deux fichiers distincts (ATP pour les hommes, WTA pour les femmes). Le traitement de ce jeu de données a nécessité l'ajout d'une colonne de genre et la concaténation des deux fichiers via Pandas avant de procéder à l'extraction. Ces traitements spécifiques garantissent que, in fine, toutes les données convergent vers un modèle unique et exploitable par notre application.